Maya Mriga - The Phantom Trail

Pinaka CTF | web | medium | 2026-11-01

Maya Mriga - The Phantom Trail

Flag: PINAKA{m4richa_illus10n_c0llaps3s_wh3n_y0u_f0ll0w_th3_cha1n}

Summary

MayaNet is a telemetry service tracking a "Golden Deer" whose telemetry id is withheld. Every event is signed with sha256(f"{x}|{y}|{prev_sig}") - a hash chain that binds only position and the previous signature, never deer_id, seq, or ts. The solve: find the one deer whose chain is rooted at the all-zero genesis, follow the prev_sig -> sig links to recover its authoritative 256-point trail (every off-chain event is a decoy "illusion"), demonstrate read-model divergence to earn a proof_token, then replay the trail to /api/validate.

Step 1 - Recon

The bundle has the offline capture capture-24h.jsonl (50,000 signed events), the official client ./mayanet-cli, and credentials ranger / jatayu-watch. The client exposes the live API surface:

- /api/update - push {seq, ts, retry, x, y} into a sandbox deer.
- /api/projection - last position ordered by (ts, retry).
- /api/authoritative - last position ordered by (seq).
- /api/proof - issues a proof_token only once the two read models disagree.
- /api/validate - {deer_id, proof_token, sequence:[[x,y],...]} -> flag if the trail matches the authoritative identity.
- /api/events/raw - the same 50,000 events as the offline capture.



figure: recon (screenshots/01_recon.png)

Step 2 - Analysis: the signature is a lure

Reversing the signature against the capture shows sig == sha256(f"{x}|{y}|{prev_sig}") for all 50,000 events - it is keyless and authenticates neither identity nor ordering. Grouping by deer reveals the tell: 8 deer carry ~6,200 events each with random genesis prev_sig values (pure noise), while one deer, deer-d51269dc11cc, has a genesis prev_sig of 00...00. Following its prev_sig -> sig links from zero yields a unique path of exactly 256 positions; its other 140 events are off-chain forks - the phantom's "illusion."



figure: analysis (screenshots/02_analysis.png)

Step 3 - Solve

solve.py rebuilds the authoritative trail from the capture, forces the two read models to diverge (one update with the highest seq, another with the highest ts, pointing at different coordinates) to obtain a proof_token, then validates the trail for deer-d51269dc11cc.

```
#!/home/kali/.ctf-tools/venv/bin/python
import json, hashlib, time, urllib.request, urllib.error
from collections import defaultdict

URL         = "https://pinaka-golden-deer.onrender.com"
USER, PW    = "ranger", "jatayu-watch"
GOLDEN_DEER = "deer-d51269dc11cc"          # the only chain rooted at the 00..00 genesis
CAPTURE     = "capture-24h.jsonl"

def call(path, token=None, method="GET", body=None, retries=5):
    data = json.dumps(body).encode() if body is not None else None
    for attempt in range(retries):
        req = urllib.request.Request(URL + path, data=data, method=method)
        req.add_header("Content-Type", "application/json")
        if token:
            req.add_header("X-Session", token)
        try:
            with urllib.request.urlopen(req, timeout=30) as r:
                return json.loads(r.read().decode())
```

```
        except (urllib.error.URLError, TimeoutError):
            if attempt == retries - 1:
                raise
            time.sleep(2 + 2 * attempt)    # free-tier host / transient DNS

def recover_trail(path):
    events = [json.loads(l) for l in open(path)]
    def sig(e):
        return hashlib.sha256(f"{e['x']}|{e['y']}|{e['prev_sig']}".encode()).hexdigest()
    assert all(sig(e) == e["sig"] for e in events)         # keyless: binds only x,y,prev_sig
    by_prev = defaultdict(list)
    for e in events:
        if e["deer_id"] == GOLDEN_DEER:
            by_prev[e["prev_sig"]].append(e)
    cur, chain = "0" * 64, []
    while cur in by_prev:
        e = by_prev[cur][0]                                # unique next; forks are decoys
        chain.append(e); cur = e["sig"]
    return [[e["x"], e["y"]] for e in chain]

def main():
    trail = recover_trail(CAPTURE)
    print(f"[*] recovered authoritative trail: {len(trail)} points")
    token = call("/api/login", method="POST", body={"username": USER, "password": PW})["token"]
    # authoritative(max seq)->(111,111) vs projection(max ts)->(222,222): divergence
    call("/api/update", token, "POST", {"seq": 100, "ts": 100,  "retry": 0, "x": 111, "y": 111})
    call("/api/update", token, "POST", {"seq": 50,  "ts": 9999, "retry": 0, "x": 222, "y": 222})
    proof_token = call("/api/proof", token)["proof_token"]
    res = call("/api/validate", token, "POST",
               {"proof_token": proof_token, "deer_id": GOLDEN_DEER, "sequence": trail})
    print(f"[+] FLAG: {res['flag']}")

if __name__ == "__main__":
    main()
```

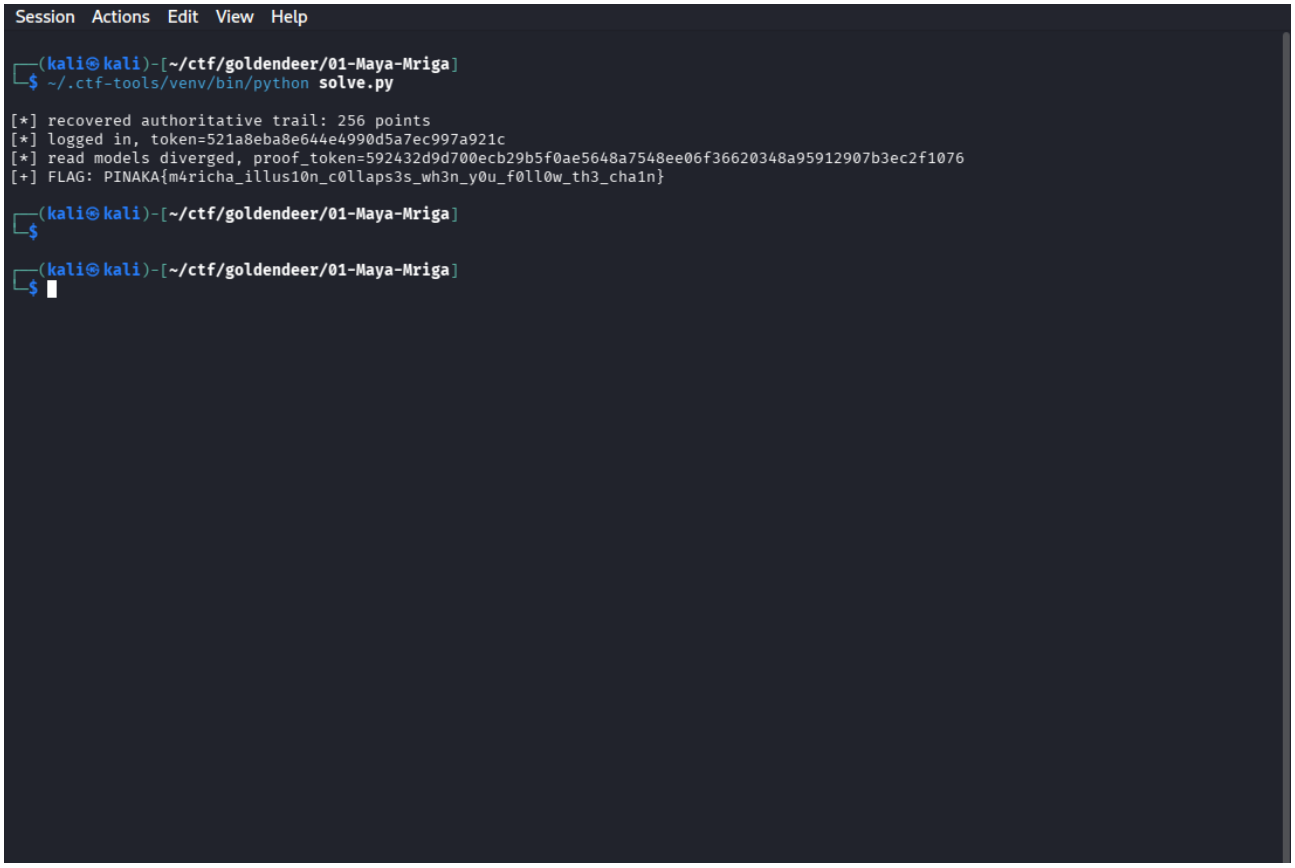


figure: solve (screenshots/03_solve.png)

Step 4 - Verify (official client only)

verify.sh reproduces the whole chain using only the provided ./mayanet-cli: login -> two divergent updates -> /api/proof (proof_token issued) -> /api/validate with the generated trail.json. The live validator returns ok: true and the same flag, confirming the solve is not an artifact of our custom script.



figure: verify (screenshots/04_verify_client.png)

Step 5 - Get the flag

The validator's JSON response and the saved flag.txt:

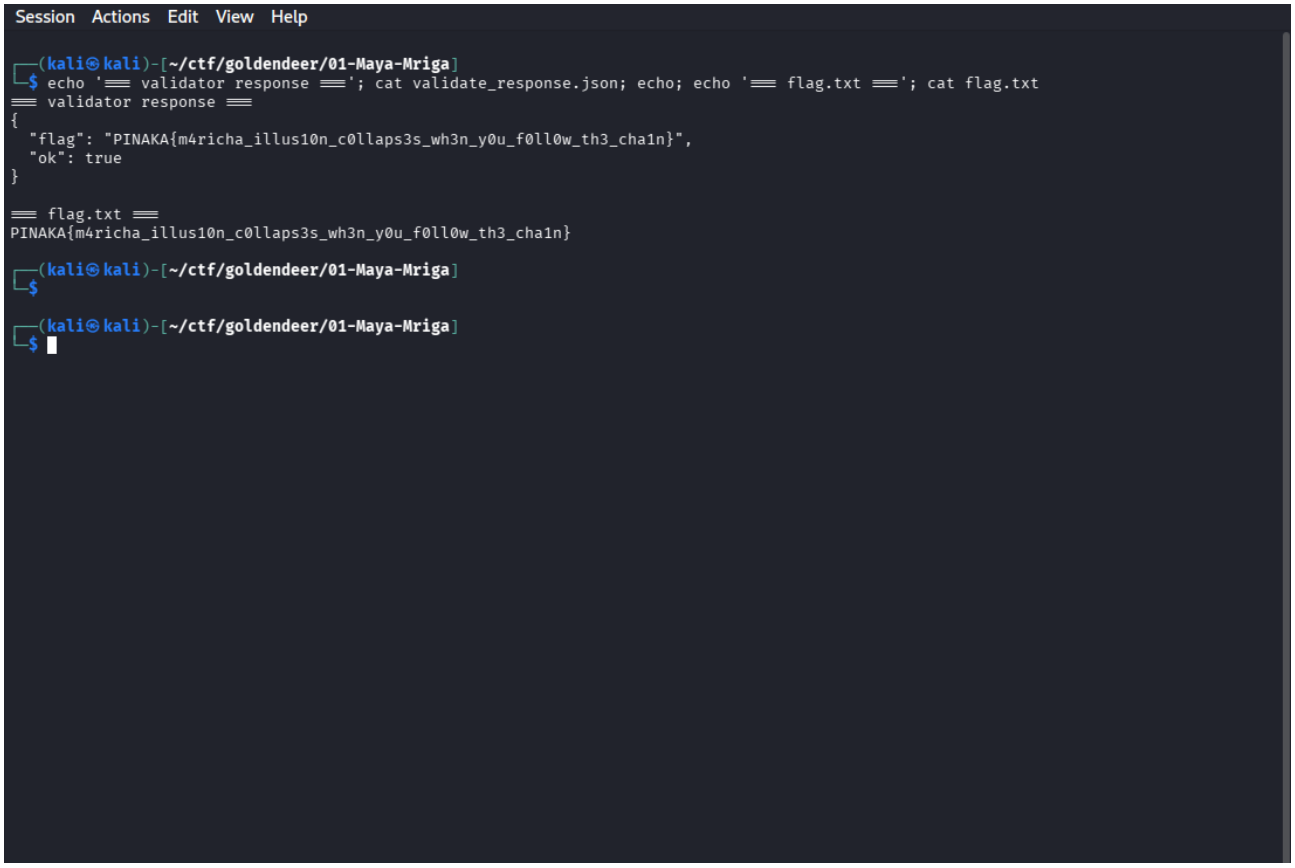


figure: flag (screenshots/05_flag.png)

Flag

```
PINAKA{m4richa_illus10n_c0llaps3s_wh3n_y0u_f0ll0w_th3_cha1n}
```

Files

File Purpose
`capture-24h.jsonl` Provided: 50,000 signed telemetry events (24h offline capture)
`mayanet-cli` Provided: official telemetry client
`analyze.py` Offline analysis - proves the keyless signature + finds the zero-genesis deer
`make_trail.py` Writes the recovered 256-point authoritative trail to `trail.json`
`solve.py` Primary end-to-end solve -> prints the flag
`verify.sh` Independent verification using only `./mayanet-cli` -> writes `flag.txt`
`trail.json` Recovered Golden Deer trail (`[[x,y],...]`)
`validate_response.json` Live validator response containing the flag
`flag.txt` The final flag
`screenshots/01-05_*.png` Screenshots of each solve step
`build_pdf.py` Builds the single self-contained PDF
`Maya-Mriga_writeup.pdf` This write-up + all screenshots, one file